

## AWS Cost Monitoring & Budget Alerts

**Services:** AWS Budgets, Cost Explorer, Lambda, SES, EventBridge

**Description:** This project builds an automated AWS cost monitoring system that continuously tracks cloud expenses and sends alerts when costs exceed a predefined limit. It uses AWS Lambda to fetch billing data from Cost Explorer, compares it with a set threshold, and sends email notifications through Amazon SES. Amazon EventBridge schedules the Lambda function to run at regular intervals, ensuring continuous and automatic monitoring. This solution offers a more flexible and customizable approach compared to AWS Budgets.

**Skills:** Cost monitoring, billing alerts, serverless automation, Lambda scripting, AWS service integration, email notification systems

**Definition:** The AWS Cost Monitoring & Budget Alerts system is a fully automated and customizable solution for tracking AWS usage costs. It uses AWS Lambda to periodically fetch cost data from Cost Explorer and compare it with a predefined limit. If the cost exceeds the threshold, an alert email is sent using Amazon SES. EventBridge ensures the process runs at scheduled intervals, enabling continuous monitoring. This approach provides better flexibility and control compared to AWS Budgets.

**Scope of the Project:** The scope of this project is to design and implement a fully automated AWS cost monitoring system that helps prevent unexpected billing by proactively notifying users when spending exceeds defined limits.

### This project covers:

- **Automated Cost Monitoring:**  
Continuously tracks AWS usage costs using Cost Explorer API.
- **Threshold-Based Alerts:**  
Sends notifications when cost exceeds a predefined budget limit.
- **Serverless Automation:**  
Uses AWS Lambda to eliminate the need for manual monitoring.
- **Scheduled Execution:**  
EventBridge triggers the monitoring process at regular intervals.
- **Email Notification System:**  
Uses Amazon SES to deliver alert messages to users.

### Methodology:

#### 1. Lambda + Cost Explorer + SES – Best Method

- Uses AWS Lambda, Cost Explorer, SES, and EventBridge for automation
- Fetches real-time cost data and compares it with a defined threshold
- Sends alert emails automatically when the cost exceeds the limit
- EventBridge schedules execution for continuous monitoring
- Fully automated, customizable, and suitable for production use

## 2. AWS Budgets – Alternative Method

- Uses AWS Budgets to define cost limits and send alerts
- Easy to set up and does not require coding
- Limited customization compared to Lambda-based solution
- Less flexible for advanced monitoring and automation

### AWS Services used in the Project:

#### 1. AWS Budgets

- Definition: AWS Budgets is a service that allows users to set custom cost and usage limits for AWS resources.
- Purpose: Helps monitor overall AWS spending.
- Role: Used as a reference for defining budget thresholds and comparison values.

#### 2. AWS Cost Explorer

- Definition: AWS Cost Explorer is a tool that provides detailed insights into AWS billing data and usage patterns.
- Purpose: Fetches real-time and historical cost data.
- Role: Used by Lambda to retrieve current cost information via API.

#### 3. AWS Lambda

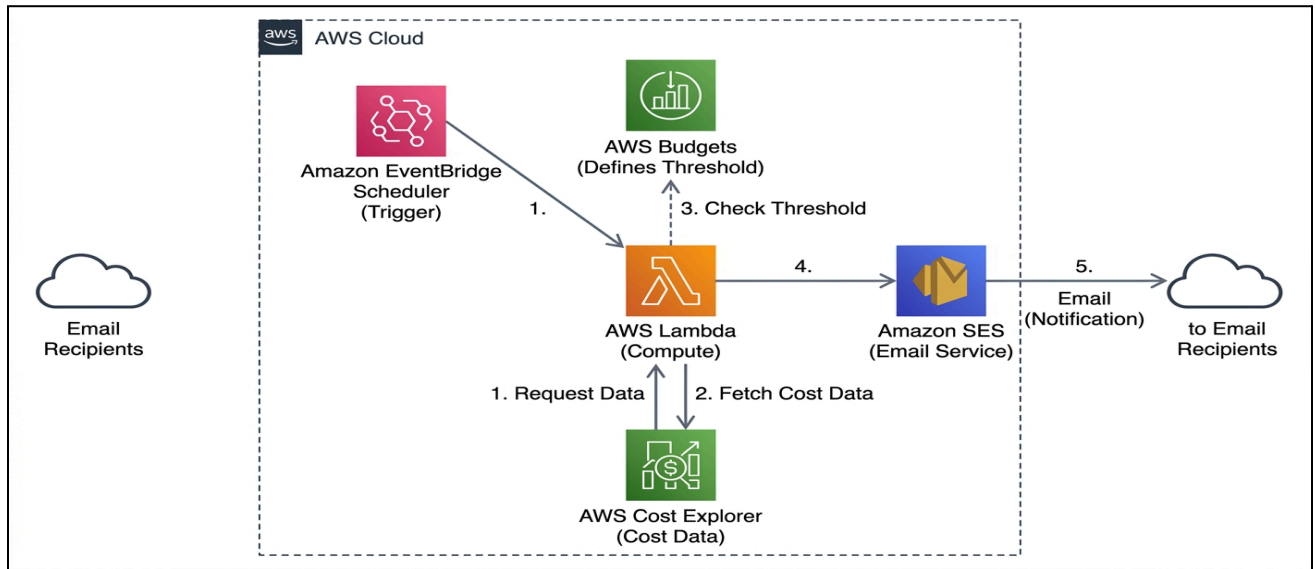
- Definition: AWS Lambda is a serverless compute service that runs code without provisioning or managing servers.
- Purpose: Automates the cost monitoring process.
- Role: Fetches cost data from Cost Explorer ,Compares cost with threshold ,Triggers email alerts when needed

#### 4. Amazon SES (Simple Email Service)

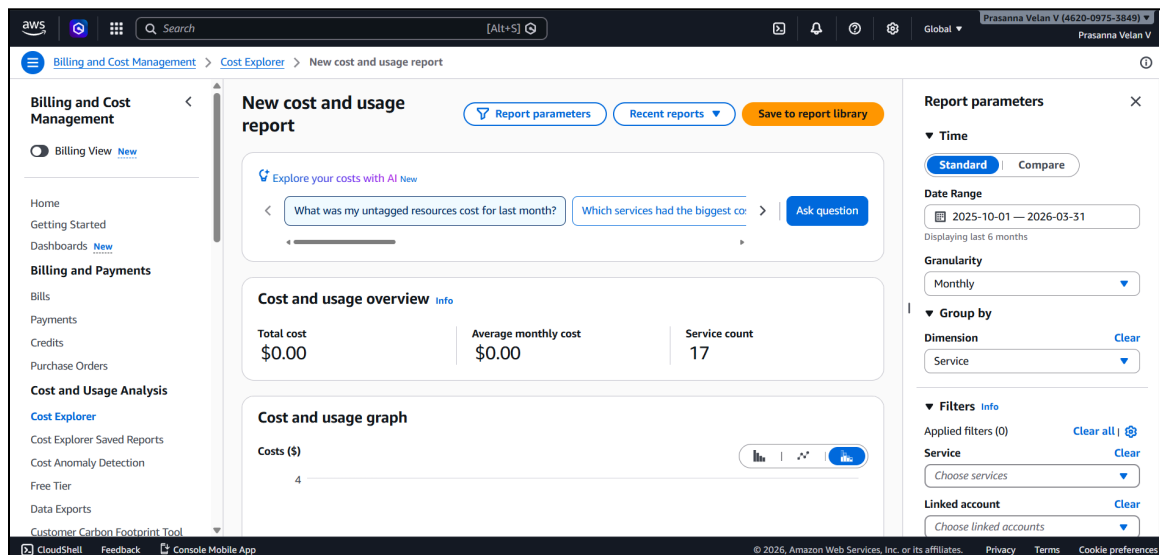
- Definition: Amazon SES is a cloud-based email sending service used for notifications and communication.
- Purpose: Sends alert emails to users.
- Role: Delivers cost alert notifications when spending exceeds limits.

#### 5. Amazon EventBridge

- Definition: Amazon EventBridge is a service that enables event-driven automation using rules and schedules.
- Purpose: Automates execution of monitoring tasks.
- Role: Triggers the Lambda function at regular intervals for continuous monitoring.

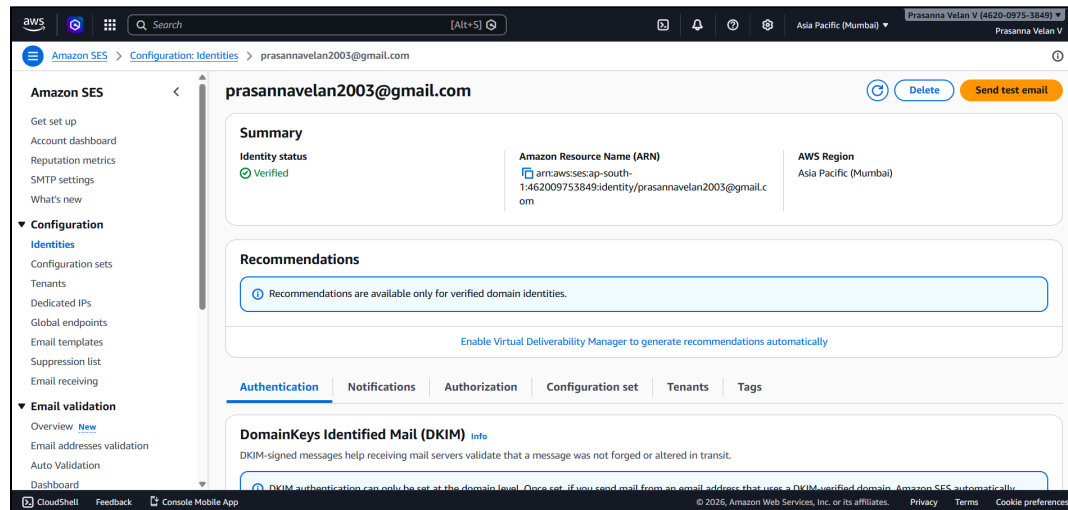
**Architecture Diagram:****Implementation Steps to Do the Project:****Step 1: Enable Cost Explorer/Budget**

- Go to AWS Console → Cost Explorer
- Click Enable Cost Explorer
- Wait for billing data to become available

**Step 2: Setup Amazon SES**

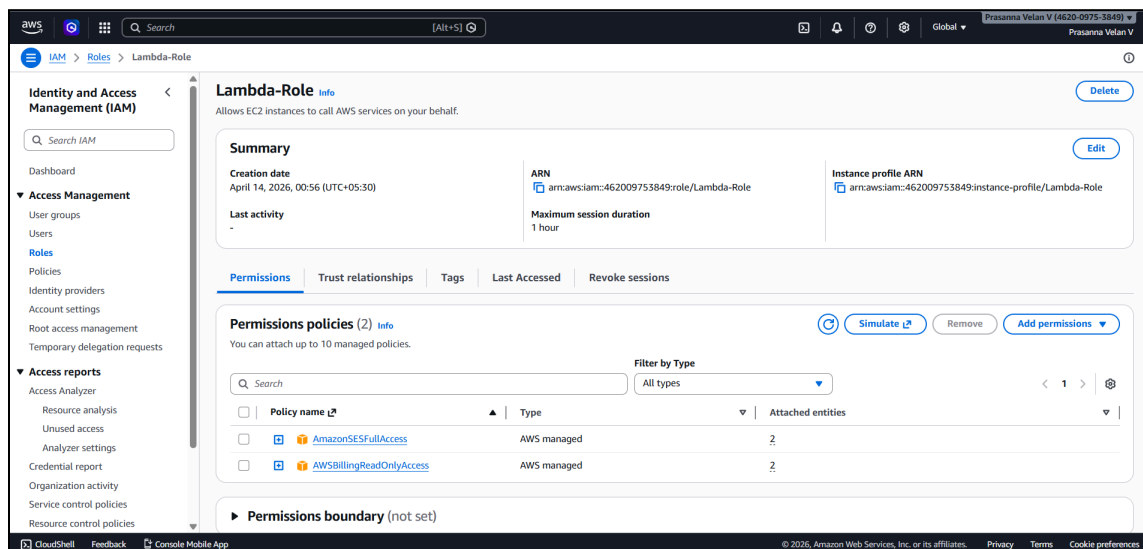
- Go to SES → Verified Identities
- Verify your email address
- Confirm verification through email

- (Optional) Request production access to send emails outside sandbox



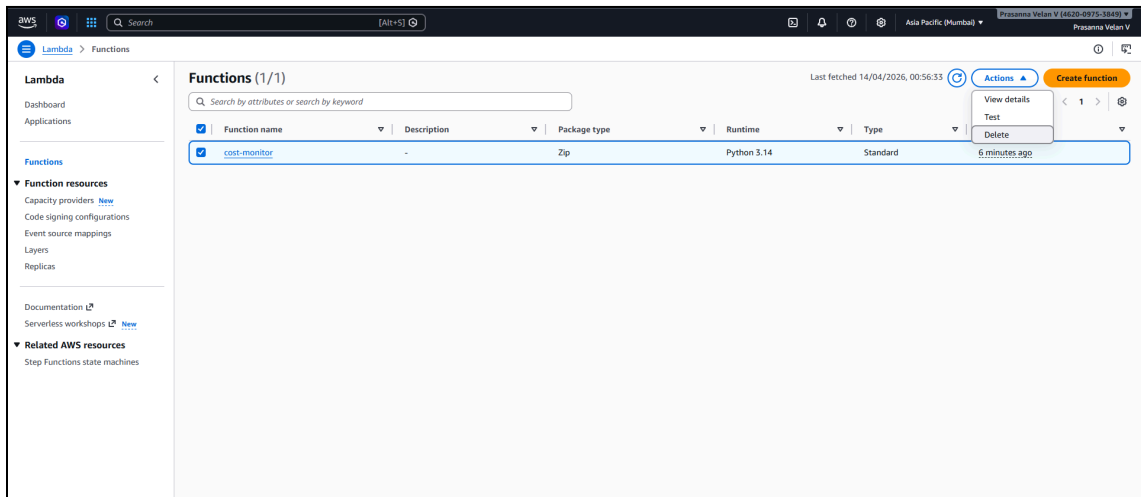
### Step 3: Create IAM Role for Lambda

- Go to IAM → Roles → Create Role → Lambda
- Attach required policies: 1. CostExplorerReadOnlyAccess , 2. AmazonSESFullAccess
- Name the role: LambdaCostMonitorRole



### Step 4: Create Lambda Function

- Go to Lambda → Create Function → Author from scratch
- Function Name: CostMonitorFunction
- Runtime: Python 3.x
- Attach IAM Role: LambdaCostMonitorRole



### Step 5: Add Lambda Code and Test

- Add Python script to: Fetch cost from Cost Explorer ,Compare with threshold and Send email using SES

#### AWS Lambda Cost Monitoring Function - Lambda code

```

import boto3
from datetime import datetime

def lambda_handler(event, context):
    print("Lambda execution started")

    try:
        ce = boto3.client('ce')
        ses = boto3.client('ses', region_name='ap-south-1')

        today = datetime.today()
        start = today.replace(day=1).strftime('%Y-%m-%d')
        end = today.strftime('%Y-%m-%d')

        print(f"Checking cost from {start} to {end}")

        response = ce.get_cost_and_usage(
            TimePeriod={
                'Start': start,
                'End': end
            },
            Granularity='MONTHLY',
            Metrics=['UnblendedCost']
        )

        cost = float(response['ResultsByTime'][0]['Total']['UnblendedCost'])

        print("Current AWS Cost:", cost)
        if cost >= 0:
            print("Preparing to send email...")

            response = ses.send_email(
                Source='prasannavelan2003@gmail.com',
                Destination={
                    'ToAddresses': ['prasannavelan2003@gmail.com']
                },
                Message={
                    'Subject': {
                        'Data': 'AWS Cost Alert'
                    },
                    'Body': {
                        'Text': {
                            'Data': f'Your current AWS cost is ${cost}'
                        }
                    }
                }
            )

            print('Email sent successfully', response['MessageId'])
            return {
                'statusCode': 200,
                'body': f'Cost checked successfully: ${cost}'
            }

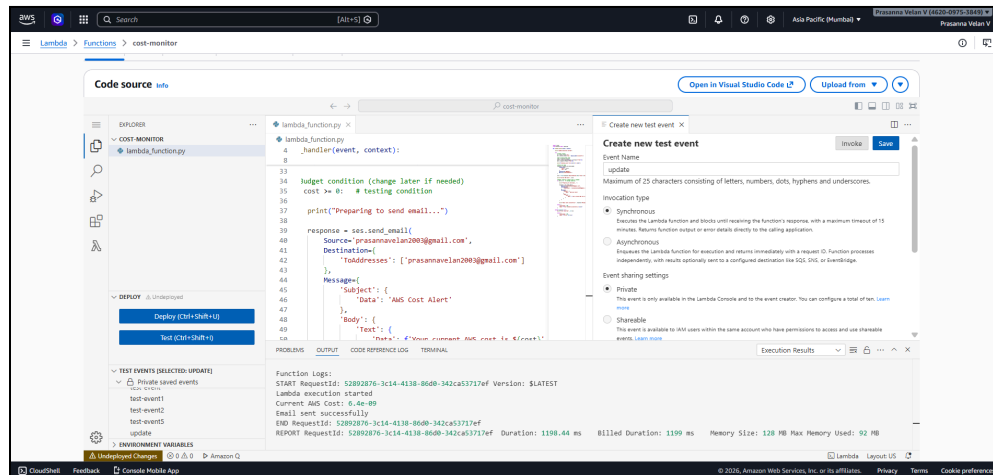
    except Exception as e:
        print('Error occurred', str(e))
        return {
            'statusCode': 500,
            'body': f'Error: {str(e)}'
        }

```

The code is annotated with numbered steps:

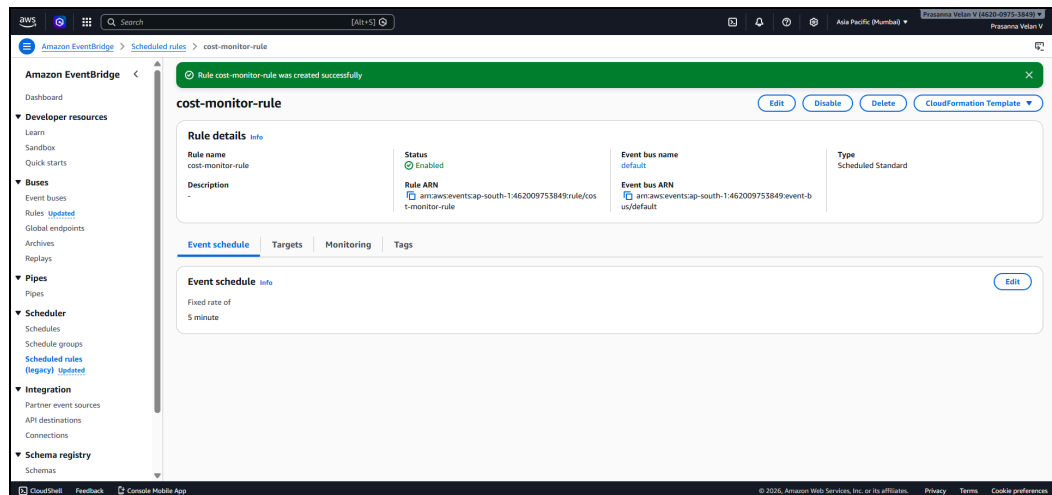
1. Initialization and Context
2. Date Range Definition
3. Cost Explorer API Call
4. Cost Calculation
5. SES Email Alert
6. Response and Exception Handling

- Click Test → Create Test Event
- Run the function and Verify: Email received



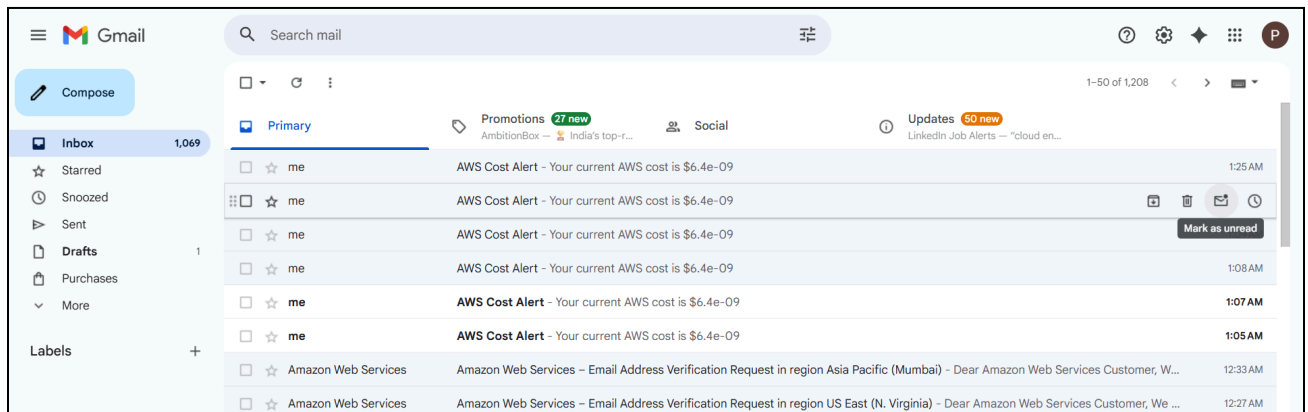
## Step 6: Schedule Lambda Using EventBridge

- Go to EventBridge → Create Rule
- Choose Schedule-based rule and Set schedule (Example: every 5 minutes) and Add target: Lambda Function (CostMonitorFunction)
- Enable rule



## Step 7: Verify Alerts

- Set a low cost threshold for testing and Wait for scheduled execution
- Confirm email alert is triggered



## Troubleshooting:

### Issue 1: Email Not Sending

- Cause: SES email not verified or still in sandbox
- Solution: Verify email address , Move SES out of sandbox if required

### Issue 2: Lambda Not Triggering

- Cause: EventBridge rule misconfiguration
- Solution: Check rule schedule , Ensure Lambda is set as target ,Enable the rule

### Issue 3: Incorrect Cost Data

- Cause: Cost Explorer data delay (can take few hours)
- Solution: Wait for updated billing data , Adjust time range in API request

### Issue 4: Multiple Alert Emails

- Cause: Frequent execution with low threshold
- Solution: Increase threshold , Reduce trigger frequency

## Conclusion:

The AWS Cost Monitoring & Budget Alerts system provides a reliable and automated solution for tracking AWS expenses and preventing unexpected billing. By integrating Lambda, Cost Explorer, SES, and EventBridge, the system continuously monitors costs and sends alerts when thresholds are exceeded.

This serverless approach eliminates manual monitoring, reduces operational effort, and provides high flexibility for customization. Compared to AWS Budgets, it offers better control over alert logic and can be extended for advanced cost management use cases.

The solution is scalable, cost-effective, and suitable for real-world deployment in organizations to maintain efficient cloud cost governance.